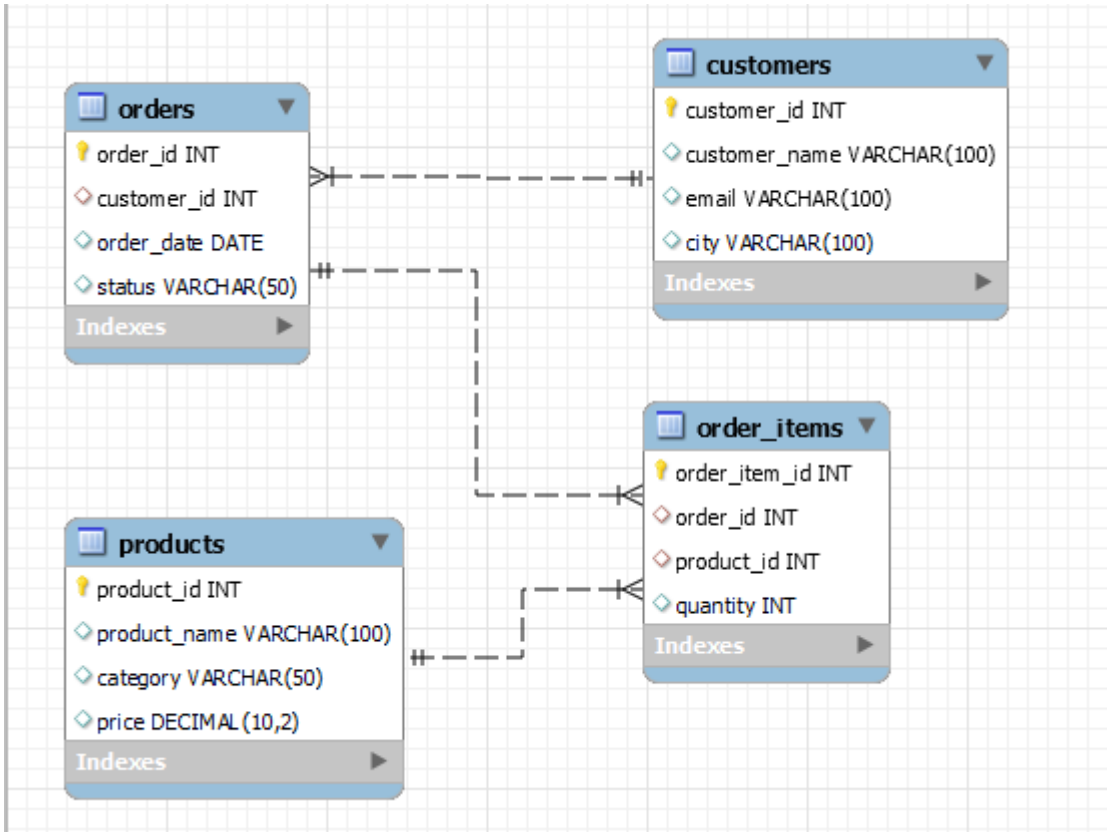
ECommerce View Assignment



Customers Table

```
5 •   select * from ecommerce_view_demo.customers;
```

Result Grid | Filter Rows: | Edit: | Export/In

customer_id	customer_name	email	city
1	Aung Aung	aung@gmail.com	Yangon
2	Su Su	susu@gmail.com	Mandalay
3	Kyaw Kyaw	kyaw@gmail.com	Yangon
NULL	NULL	NULL	NULL

Orders_items Table

```
5 • select * from ecommerce_view_demo.order_items;
```

Result Grid | Filter Rows: | Edit: | Export/Im

order_item_id	order_id	product_id	quantity
1	1	1	1
2	1	2	2
3	2	3	1
4	3	2	1
NULL	NULL	NULL	NULL

Orders Table

```
5 • select * from ecommerce_view_demo.orders;
```

Result Grid | Filter Rows: | Edit:

order_id	customer_id	order_date	status
1	1	2024-01-10	Completed
2	2	2024-01-11	Pending
3	1	2024-01-12	Completed
NULL	NULL	NULL	NULL

Products Table

```
5 • select * from ecommerce_view_demo.products;
```

Result Grid | Filter Rows: | Edit:

product_id	product_name	category	price
1	Laptop	Electronics	1500000.00
2	Phone	Electronics	800000.00
3	Shoes	Fashion	120000.00
NULL	NULL	NULL	NULL

Q1: Create a VIEW to Show Customer Basic Information (Hide Email)

```
71 •  CREATE VIEW vw_customer_info AS
72    SELECT customer_id,
73           customer_name,
74           city
75    FROM   customers;
76 •  Select * from vw_customer_info;
```

Result Grid | Filter Rows: | Export:

customer_id	customer_name	city
1	Aung Aung	Yangon
2	Su Su	Mandalay
3	Kyaw Kyaw	Yangon

Q2: Create a VIEW to Show All Completed Orders

```
79    -- Q2 :  Create a VIEW to Show All Completed Orders
80 •  CREATE VIEW vw_completed_orders AS
81    SELECT o.order_id,
82           c.customer_name,
83           o.order_date,
84           o.status
85    FROM   orders    o
86    JOIN   customers c ON o.customer_id = c.customer_id
87    WHERE  o.status = 'Completed';
88
89 •  SELECT * FROM vw_completed_orders;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

order_id	customer_name	order_date	status
1	Aung Aung	2024-01-10	Completed
3	Aung Aung	2024-01-12	Completed

Q3: Create a JOIN VIEW for Order Details (Customer + Product)

```
90      -- Q3: Create a JOIN VIEW for Order Details (Customer + Product)
91 •    CREATE VIEW vw_order_details AS
92      SELECT c.customer_name,
93             o.order_id,
94             o.order_date,
95             p.product_name,
96             p.price,
97             oi.quantity,
98             (p.price * oi.quantity) AS line_total
99      FROM   customers   c
00      JOIN   orders      o  ON c.customer_id = o.customer_id
01      JOIN   order_items oi ON o.order_id    = oi.order_id
02      JOIN   products    p  ON oi.product_id = p.product_id;
03 •    SELECT * FROM vw_order_details;
```

esult Grid | Filter Rows: | Export: | Wrap Cell Content:

customer_name	order_id	order_date	product_name	price	quantity	line_total
Aung Aung	1	2024-01-10	Laptop	1500000.00	1	1500000.00
Aung Aung	1	2024-01-10	Phone	800000.00	2	1600000.00
Su Su	2	2024-01-11	Shoes	120000.00	1	120000.00
Aung Aung	3	2024-01-12	Phone	800000.00	1	800000.00

Q4: Create a VIEW to Calculate Total Sales per Order

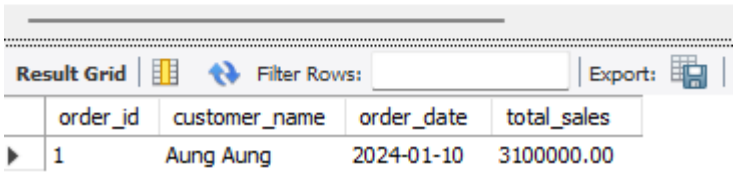
```
106 •   CREATE VIEW vw_order_totals AS
107     SELECT o.order_id,
108            c.customer_name,
109            o.order_date,
110            SUM(p.price * oi.quantity) AS total_sales
111     FROM   orders      o
112     JOIN   customers   c  ON o.customer_id = c.customer_id
113     JOIN   order_items oi ON o.order_id    = oi.order_id
114     JOIN   products    p  ON oi.product_id = p.product_id
115     GROUP BY o.order_id, c.customer_name, o.order_date;
116 •   SELECT * FROM vw_order_totals;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

order_id	customer_name	order_date	total_sales
1	Aung Aung	2024-01-10	3100000.00
2	Su Su	2024-01-11	120000.00
3	Aung Aung	2024-01-12	800000.00

Q5: Create a VIEW for High-Value Orders (Above 1,000,000)

```
119 •   CREATE VIEW vw_high_value_orders AS
120     SELECT order_id,
121            customer_name,
122            order_date,
123            total_sales
124     FROM   vw_order_totals
125     WHERE  total_sales > 1000000;
126 •   SELECT * FROM vw_high_value_orders;
```



Result Grid | Filter Rows: | Export:

order_id	customer_name	order_date	total_sales
1	Aung Aung	2024-01-10	3100000.00

Q6: Create a VIEW with WITH CHECK OPTION

-- VALID: status = 'Completed' -- succeeds

INSERT INTO vw_completed_only (customer_id, order_date, status) VALUES (2, '2024-02-01', 'Completed');

-- INVALID: status = 'Pending' -- FAILS

INSERT INTO vw_completed_only (customer_id, order_date, status) VALUES (3, '2024-02-02', 'Pending'); -- ERROR 1369 (HY000): CHECK OPTION failed 'ecommerce_view_demo.vw_completed_only'

Q7: Try to Update Data Using VIEW (Updatable View)

```
142     -- Q7: Try to Update Data Using VIEW (Updatable View)
143     -- vw_customer_info is updatable (single table, no GROUP BY, no aggregation)
144 •   UPDATE vw_customer_info
145     SET    city = 'Naypyidaw'
146     WHERE  customer_id = 1;
147     -- Verify change in base table
148 •   SELECT customer_id, customer_name, city FROM customers WHERE customer_id = 1
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell C

customer_id	customer_name	city
1	Aung Aung	Naypyidaw
NULL	NULL	NULL

```
151     -- Also visible in the view
152 •   SELECT * FROM vw_customer_info WHERE customer_id = 1;
153     -- Result: 1 | Aung Aung | Naypyidaw
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customer_id	customer_name	city
1	Aung Aung	Naypyidaw

Q8: Try Invalid Update (Should Fail)

```
4     -- Q8: Try Invalid Update (Should Fail)
5     -- FAIL: vw_order_totals has GROUP BY + SUM
6 •   UPDATE vw_order_totals
7     SET    total_sales = 9999999
8     WHERE  order_id = 1;
9     -- ERROR 1288 (HY000): The target table vw_order_totals of the UPDATE is not updatable
0     -- FAIL: vw_order_details joins 4 tables
1 •   UPDATE vw_order_details
2     SET    product_name = 'MacBook'
3     WHERE  order_id = 1;
4     -- ERROR 1288 (HY000): The target table vw_order_details of the UPDATE is not updatable
5
6
7
```

Q9: Create a VIEW for Customer Purchase Summary

```
66      -- Q9: Create a VIEW for Customer Purchase Summary
67 •    CREATE VIEW vw_customer_summary AS
68      SELECT c.customer_id,
69             c.customer_name,
70             c.city,
71             COUNT(DISTINCT o.order_id)   AS total_orders,
72             SUM(oi.quantity)             AS total_items,
73             SUM(p.price * oi.quantity)   AS total_spent
74      FROM   customers   c
75      JOIN   orders      o  ON c.customer_id = o.customer_id
76      JOIN   order_items oi ON o.order_id    = oi.order_id
77      JOIN   products    p  ON oi.product_id = p.product_id
78      GROUP BY c.customer_id, c.customer_name, c.city;
79 •    SELECT * FROM vw_customer_summary;
80
```

esult Grid | Filter Rows: | Export: | Wrap Cell Content:

customer_id	customer_name	city	total_orders	total_items	total_spent
1	Aung Aung	Naypyidaw	2	4	3900000.00
2	Su Su	Mandalay	1	1	120000.00

Q10: Create a VIEW Using ALGORITHM = TEMPTABLE

```
182     -- Q10: Create a VIEW Using ALGORITHM = TEMPTABLE
183 •   CREATE ALGORITHM = TEMPTABLE VIEW vw_sales_summary AS
184     SELECT p.category,
185            COUNT(oi.order_item_id)     AS total_items_sold,
186            SUM(p.price * oi.quantity)  AS category_revenue
187     FROM   products    p
188     JOIN   order_items oi ON p.product_id = oi.product_id
189     GROUP BY p.category;
190 •   SELECT * FROM vw_sales_summary;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

category	total_items_sold	category_revenue
Electronics	3	3900000.00
Fashion	1	120000.00

Q11: Create a VIEW for Security (Hide Price)

```
93 •  CREATE VIEW vw_product_catalog AS
94    SELECT product_id,
95           product_name,
96           category
97           -- price intentionally excluded
98    FROM   products;
99 •  SELECT * FROM vw_product_catalog;
```

esult Grid | Filter Rows: | Export: | Wrap Ce

product_id	product_name	category
1	MacBook	Electronics
2	MacBook	Electronics
3	Shoes	Fashion

Q12: Show All Views in Database

```
202    -- Method 1: Quick list
203 •  SHOW FULL TABLES IN ecommerce_view_demo WHERE TABLE_TYPE = 'VIEW';
204
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Tables_in_ecommerce_view_demo	Table_type
vw_completed_only	VIEW
vw_completed_orders	VIEW
vw_customer_info	VIEW
vw_customer_summary	VIEW
vw_high_value_orders	VIEW
vw_order_details	VIEW
vw_order_totals	VIEW
vw_product_catalog	VIEW
vw_sales_summary	VIEW

```sql
-- Method 2: Detailed with updatability
SELECT TABLE_NAME    AS view_name,
       IS_UPDATABLE
FROM   INFORMATION_SCHEMA.VIEWS
WHERE  TABLE_SCHEMA = 'ecommerce_view_demo';
```

Result Grid | Filter Rows: | Export: | Wrap Ce

view_name	IS_UPDATABLE
vw_completed_only	YES
vw_completed_orders	YES
vw_customer_info	YES
vw_customer_summary	NO
vw_high_value_orders	NO
vw_order_details	YES
vw_order_totals	NO
vw_product_catalog	YES
vw_sales_summary	NO

Q13: Show View Definition

```sql
-- Q13: Show View Definition
SHOW CREATE VIEW vw_order_details;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

View	Create View	character_set_client	collation_connection
vw_order_details	CREATE ALGORITHM=UNDEFINED DEFINER=`r...	utf8mb4	utf8mb4_0900_ai_ci

Q14: Drop a View

```sql
-- Drop a single view
DROP VIEW IF EXISTS vw_high_value_orders;
-- Drop multiple views at once
DROP VIEW IF EXISTS vw_completed_orders, vw_completed_only;
-- Verify
SHOW FULL TABLES IN ecommerce_view_demo WHERE TABLE_TYPE = 'VIE
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Tables_in_ecommerce_view_demo	Table_type
vw_customer_info	VIEW
vw_customer_summary	VIEW
vw_order_details	VIEW
vw_order_totals	VIEW
vw_product_catalog	VIEW
vw_sales_summary	VIEW

Additional Q1: Create a View for Pending Orders Only

```
227     SELECT o.order_id,
228            c.customer_name,
229            c.city,
230            o.order_date,
231            o.status
232     FROM   orders    o
233     JOIN   customers c ON o.customer_id = c.customer_id
234     WHERE  o.status = 'Pending';
235     SELECT * FROM vw_pending_orders;
236
```

Result Grid | Filter Rows: | Export: | Wrap Cell Cont

order_id	customer_name	city	order_date	status
2	Su Su	Mandalay	2024-01-11	Pending

Additional Q2: Create a View Showing Top 3 Expensive Products

```
238     CREATE VIEW vw_top3_expensive AS
239     SELECT product_id,
240            product_name,
241            category,
242            price
243     FROM   products
244     ORDER BY price DESC
245     LIMIT 3;
246     SELECT * FROM vw top3 expensive;
```

Result Grid | Filter Rows: | Export: | Wrap Cell

product_id	product_name	category	price
1	MacBook	Electronics	1500000.00
2	MacBook	Electronics	800000.00
3	Shoes	Fashion	120000.00

Additional Q3: Create a View Showing Customer Orders by City

```sql
CREATE VIEW vw_orders_by_city AS
SELECT c.city,
       COUNT(DISTINCT o.order_id)    AS total_orders,
       COUNT(DISTINCT c.customer_id) AS total_customers,
       SUM(p.price * oi.quantity)    AS city_revenue
FROM   customers   c
JOIN   orders      o  ON c.customer_id = o.customer_id
JOIN   order_items oi ON o.order_id    = oi.order_id
JOIN   products    p  ON oi.product_id = p.product_id
GROUP BY c.city
ORDER BY city_revenue DESC;
SELECT * FROM vw_orders_by_city;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

city	total_orders	total_customers	city_revenue
Naypyidaw	2	1	3900000.00
Mandalay	1	1	120000.00

Additional Q4: Create a View for Monthly Sales Report

```sql
-- Additional Q4: Create a View for Monthly Sales Report
CREATE VIEW vw_monthly_sales AS
SELECT YEAR(o.order_date)                AS sale_year,
       MONTH(o.order_date)               AS sale_month,
       DATE_FORMAT(o.order_date,'%Y-%m') AS month_label,
       COUNT(DISTINCT o.order_id)        AS total_orders,
       SUM(p.price * oi.quantity)        AS monthly_revenue
FROM   orders      o
JOIN   order_items oi ON o.order_id    = oi.order_id
JOIN   products    p  ON oi.product_id = p.product_id
GROUP BY sale_year, sale_month, month_label
ORDER BY sale_year, sale_month;
SELECT * FROM vw_monthly_sales;
```

esult Grid | Filter Rows: | Export: | Wrap Cell Content:

sale_year	sale_month	month_label	total_orders	monthly_revenue
2024	1	2024-01	3	4020000.00

Additional Q5: Create a View Showing Products Not Ordered

```sql
-- Additional Q5: Create a View Showing Products Not Ordered
CREATE VIEW vw_unordered_products AS
SELECT p.product_id,
       p.product_name,
       p.category,
       p.price
FROM   products    p
LEFT JOIN order_items oi ON p.product_id = oi.product_id
WHERE  oi.order_item_id IS NULL;
-- Currently all products have been ordered, so result is empty
SELECT * FROM vw_unordered_products;
-- Result: Empty set (0 rows)
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

product_id	product_name	category	price
4	Tablet	Electronics	600000.00

```sql
SELECT * FROM vw_unordered_products;
-- Result: Empty set (0 rows)
-- Test by adding an unordered product
INSERT INTO products (product_name, category, price) VALUES ('Tablet', 'Electronics', 600000
-- Now the view returns the new product
SELECT * FROM vw_unordered_products;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

product_id	product_name	category	price
4	Tablet	Electronics	600000.00
5	Tablet	Electronics	600000.00